

FEDERAL UNIVERSITY
OF SANTA CATARINA

Laboratório 5:
Circuitos sequenciais, latches e flip-flops

EEL5105 – Circuitos e Técnicas Digitais

Introdução

Tarefa
Emulação na placa DE2

Introdução

- **Objetivos**

1. Entender o conceito de circuitos sequenciais.
2. Explicar o uso de descrição VHDL **comportamental**.
3. Entender o conceito e diferença entre **flip-flops** e **latches**.
4. Entender descrição de **registradores em VHDL**.
5. **Implementação** de flip-flops, latches e registradores em VHDL.
6. Estudo de caso: projeto de circuito aritmético e uso de registradores como elementos de "**memória**".

Introdução

- **Descrição de circuitos sequenciais em VHDL**

```
library ieee;
use ieee.std_logic_1164.all;
entity Sinais is port (
         C, D:  in  std_logic;
         Q:     out std_logic
                    );
end Sinais;
```

```
architecture behv of Sinais is

begin
```

```
      P1: process (C, D)
      begin

Q <= 'Z';
if (C = '1') then  Q <= D;
end if;

      end process P1;

end behv;
```

Process

• Define uma SEQUÊNCIA DE COMANDOS a ser realizada pelo circuito. Um processo nunca termina - CÍCLICO.

• O processo é acionado, e sua sequência de comandos é executada, sempre que ocorrer uma alteração em algum elemento da sua LISTA DE PARÂMETROS (Ex. C ou D).

• Dessa forma, após a execução do último comando, o primeiro comando da sequência é executado, SEMPRE que ocorrer uma nova alteração em algum parâmetro.

• Obs. Comando IF .. THEN .. ELSE é utilizado APENAS dentro de um *process*. LEMBRE DE USAR IF .. THEN .. ELSE UNICAMENTE EM CIRCUITOS SEQUENCIAIS!!!!
Para usar atribuição condicional em circuitos puramente combinacionais use sempre WHEN... ELSE.

Introdução

- **Latch D**

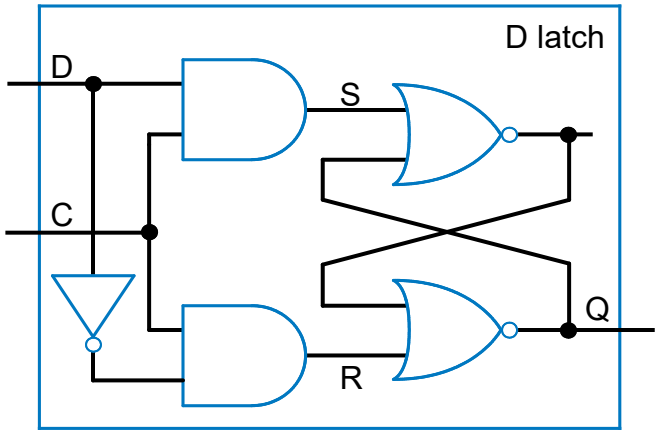
```
library ieee;
use ieee.std_logic_1164.all;

entity D_latch is port (
                          C, D:  in std_logic;
                          Q: out std_logic
                       );
end D_latch;

architecture behv of D_latch is
begin
          process(C, D)
          begin

                    if (C = '1') then Q <= D;
                    end if;

          end process;
end behv;
```



C	D	Q_{t+1}
0	X	Q_t
1	0	0
1	1	1

Mantém estado

Introdução

- **Flip-Flop D**

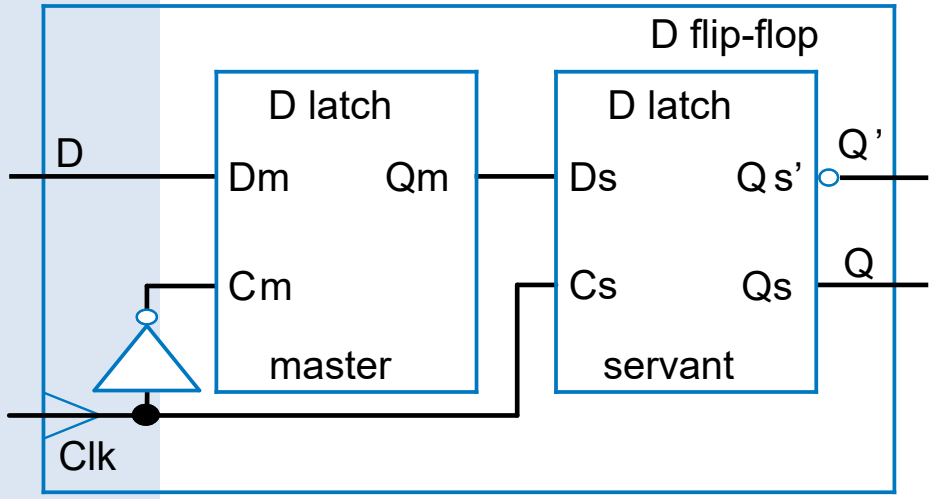
```
library ieee;
use ieee.std_logic_1164.all;

entity D_FF is port (
                        CLK, D:  in std_logic;
                        Q: out std_logic
                    );
end D_FF;

architecture behv of D_FF is
begin
    process(CLK, D)
    begin

if (CLK'event and CLK = '1') then Q <= D;
end if;

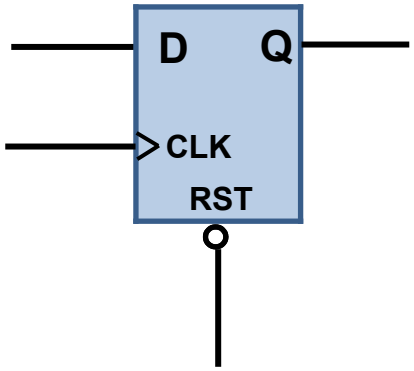
    end process;
end behv;
```



• **Se CLK mudar de '0' para '1' (borda de subida), então Q receberá o nível lógico de D.**

Introdução

- **Flip-Flop D com RESET assíncrono**



• Sempre que a entrada RST for Zero, a saída Q será Zero.

• Quando RST for diferente de Zero, o valor na saída Q vai depender da entrada CLK.

• Se CLK for de '0' para '1' (borda de subida), então Q receberá o nível lógico de D.

```
library ieee;
use ieee.std_logic_1164.all;
entity D_FF is port (
            CLK, RST, D:  in  std_logic;
            Q:            out std_logic
                        );
end D_FF;

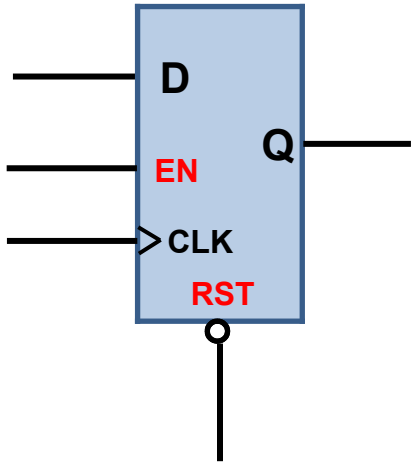
architecture behv of D_FF is
begin
    process (CLK,  RST, D)
    begin

if (RST = '0') then  Q <= '0';
elsif (CLK'event and CLK = '1') then Q <= D;
end if;

 end process;
end behv;
```

Introdução

- **Flip-Flop D com *Reset* e *Enable***



• Sempre que a entrada RST for Zero, a saída Q será Zero.

• Se CLK mudar de '0' para '1' (borda de subida) e o sinal de *Enable* (EN) estiver em '1', então Q receberá o nível lógico de D.

```
library ieee;
use ieee.std_logic_1164.all;
entity D_FF is port (
            CLK  RST, EN, D:  in  std_logic;
            Q:                out std_logic
          );
end D_FF;

architecture behv of D_FF is
begin
    process (CLK,  RST, D, EN)
    begin
              if RST = '0' then    Q <= '0';
              elsif CLK'event and CLK = '1' then
                        if EN = '1' then  Q <= D;
                        end if;
              end if;
    end process;
end behv;
```

Introdução

- ***Registrador***

```
library ieee;
use ieee.std_logic_1164.all;

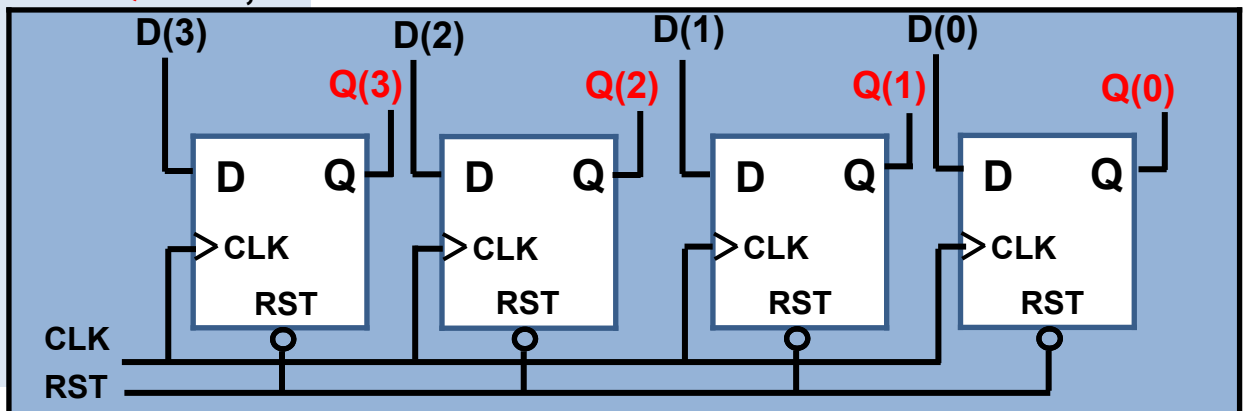
entity D_4FF is port (
        CLK, RST:  in    std_logic;
        D:              in    std_logic_vector (3 downto 0);
        Q:              out  std_logic_vector (3 downto 0)
         );
end D_4FF;
architecture behv of D_4FF is
begin
     process(CLK, D, RST)
     begin

if  RST = '0' then   Q <= "0000";
elsif (CLK'event and CLK = '1') then  Q <= D;
end if;

 end process;
end behv;
```

- Enquanto não ocorrer um novo evento no sinal CLK e enquanto esse evento for diferente de '1' (borda de subida), então a saída Q do flip-flop continuará armazenando o valor atual.

- Ao ocorrer um novo evento em CLK, e se esse novo evento for uma transição de '0' para '1' (borda de subida), então a saída Q receberá o novo valor existente na entrada D.



Introdução

Tarefa

Emulação na placa DE2

Tarefa

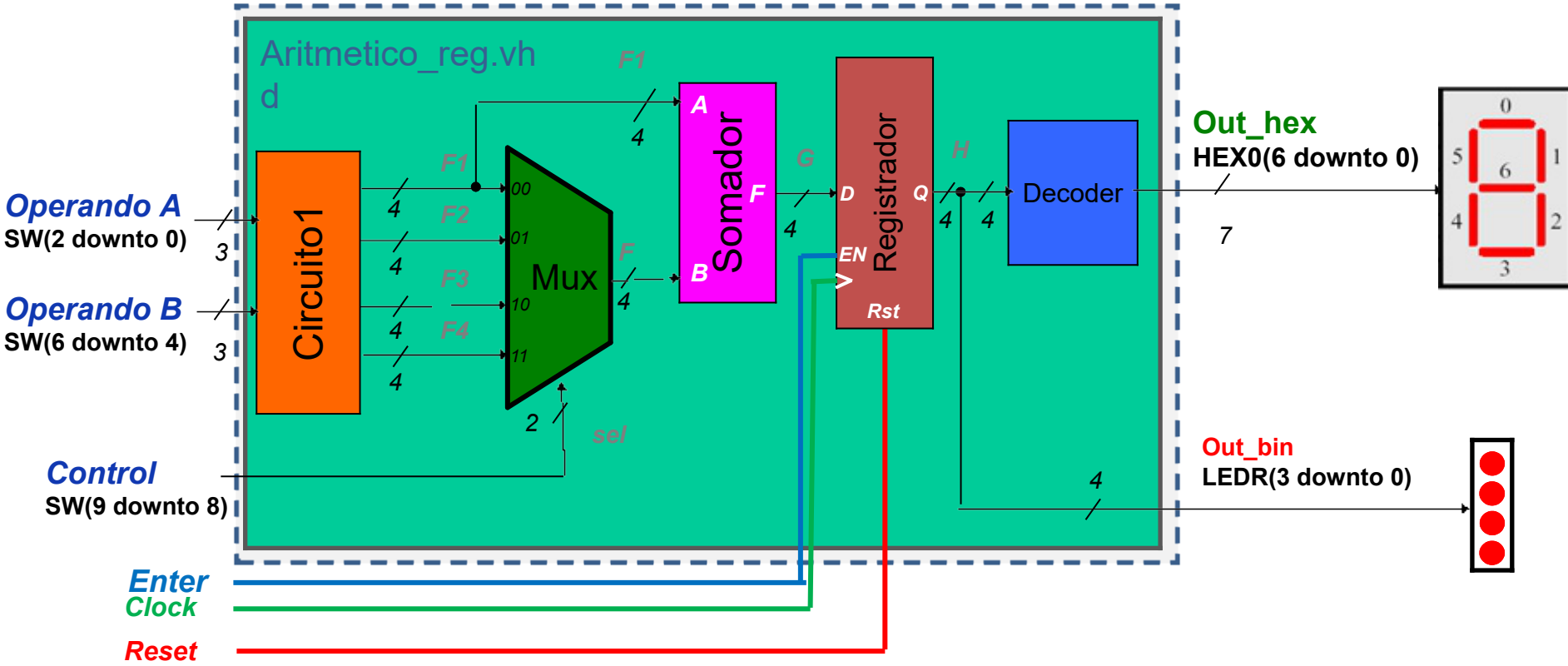
- O circuito consiste no circuito aritmetico do laboratório anterior, porém com a inclusão de um registrador para armazenar os resultados das operações.

Operação *Sel(1 downto 0)*	Operando B *B(2 downto 0)*	Operando A *A(2 downto 0)*

Seletor	Saída
00	2A
01	3A
10	A+B
11	A+2B

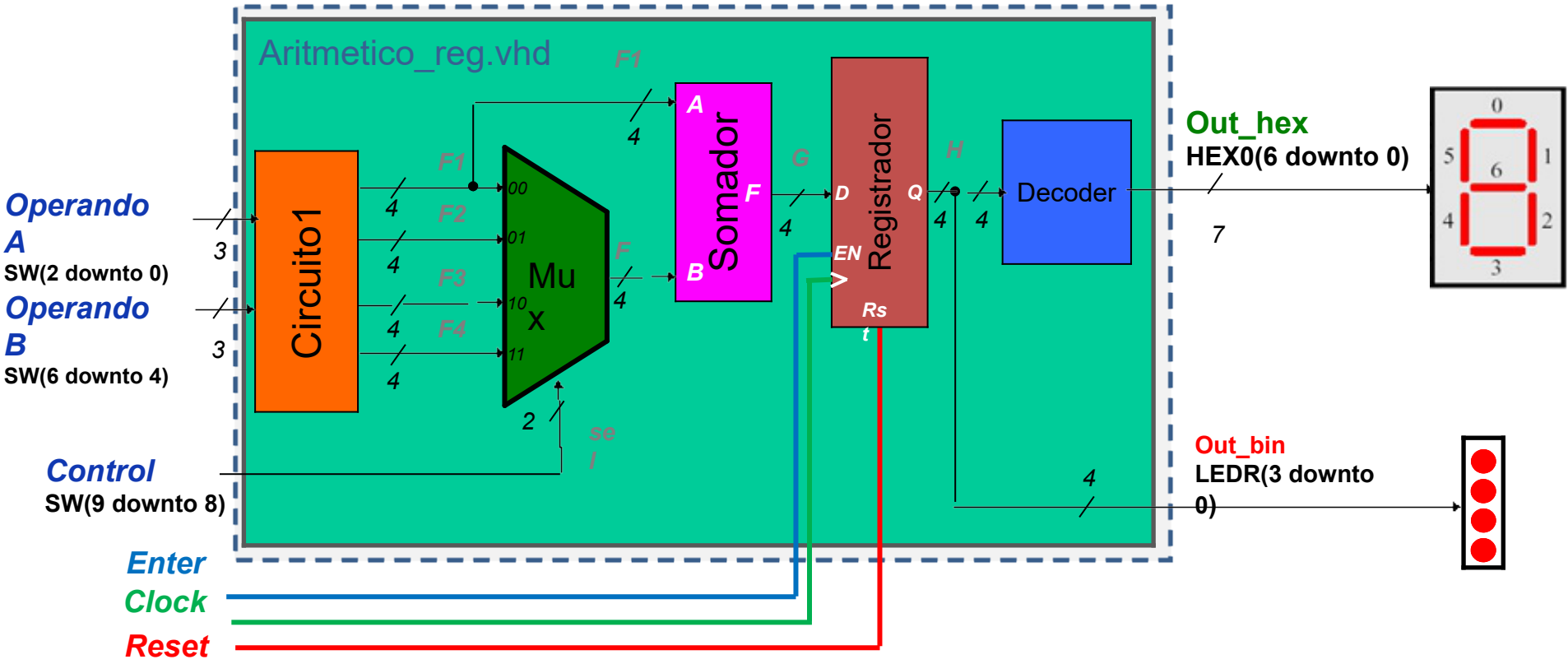
Tarefa

- Um **registrador de 4 bits** para armazenar o resultado a ser apresentado nas saídas.
- Apresentou-se no slide 9, um registrador de 4 bits implementado com 4 ***flip-flops***, porém sem **Enable**. Utilizar os circuitos dos **slides 8 e 9** como base para para escrever o VHDL do registrador de 4 bits solicitados no exercício, com **Reset** e **Enable**.



Tarefa

- Ao se pressionar o “**Reset** ”, os flip-flops devem apresentar **'0'** em suas saídas **.**
- Ao se pressionar o “**Enter**”, os flip-flops são habilitados para escrita, armazenando os valores presentes nas suas entradas (“memória”). Na **entity do aritmetico_reg**, devem ser adicionados os sinais **reset**, **enter** e **clock** (1 bit).



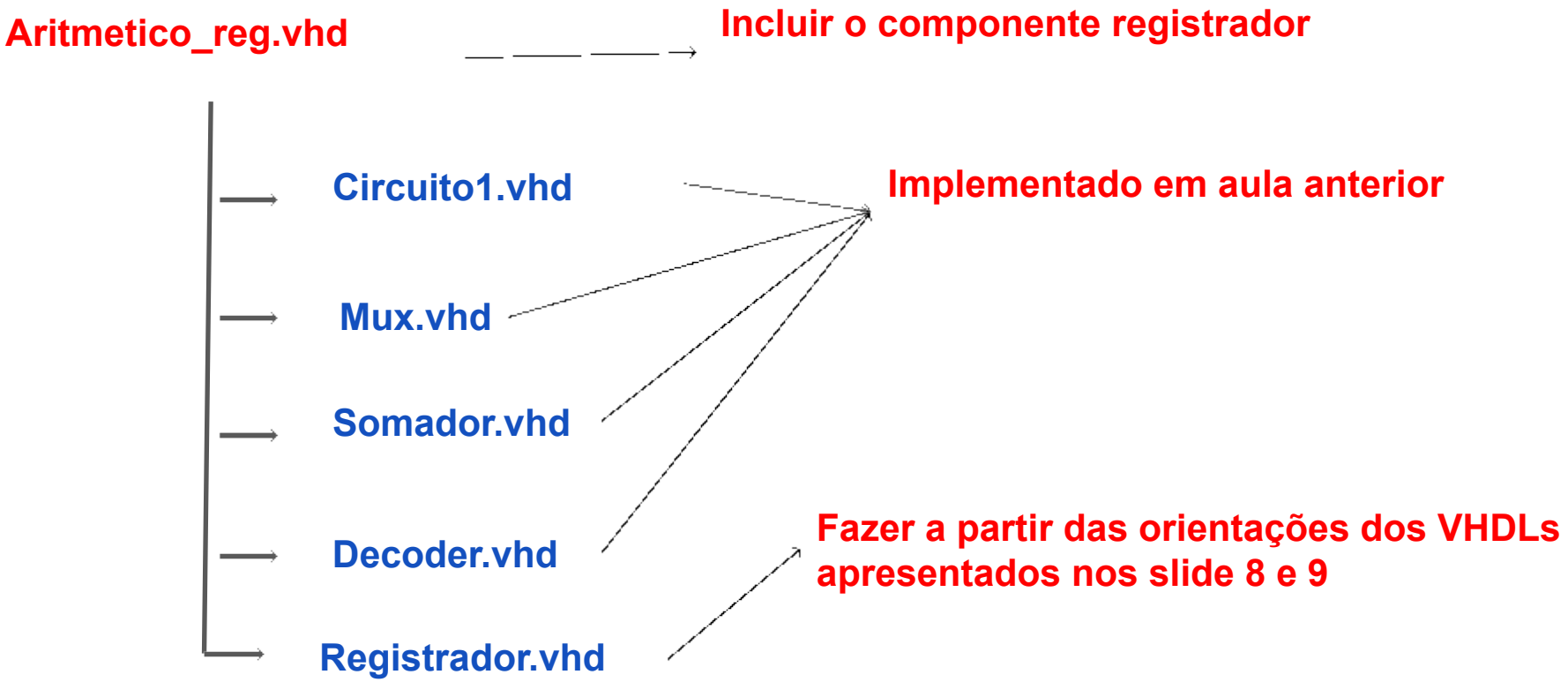
Tarefa

- **Passo 1**: Criar projeto no **Quartus II**
  - Acessar **File -> New Project Wizard** e criar um projeto com as seguintes características:
    - **Pasta**: nomear a pasta como ***AULA5/***
    - **Sugestão de nome do Projeto**: **Aritmetico_reg**
    - **Tipo**: *Empty Project*
    - Em ***Add Files***, não adicionar nenhum arquivo
    - Demais opções deixar como padrão

- **Passo 2**: Criar arquivo do tipo **VHDL** dentro do projeto
  - No **Quartus II**, acessar **File -> New -> Design Files -> VHDL File** e o arquivo "***Vhdl1.vhd***" será criado:
  - Em seguida, clique em **File -> Save As** e salve seu arquivo com algum nome desejado (sugestão: "***aritmetico_reg.vhd***").

Tarefa

- **Passo 3**: **Crie** um projeto **VHDL** que implemente o circuito apresentado no slide 13 com a seguinte hierarquia:



- **Passo 4**: **Compile** o projeto **Aritmetico_reg** e faça a **análise dos eventuais erros** ocorridos.

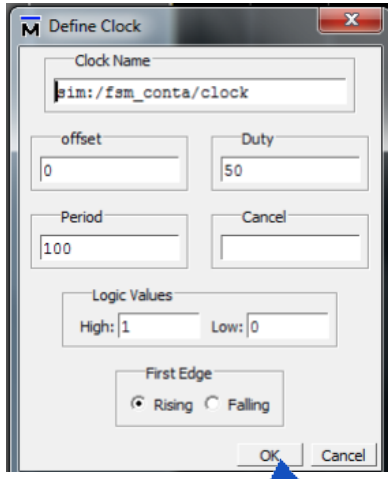
Tarefa

- **Passo 5**: Após a compilação e verificação de que não há erros de sintaxe **simular** o circuito no **ModelSim** para as 4 operações possíveis, 2A, 3A, A+B, A+2B da seguinte forma:

  - Uma vez todas as entradas e saídas na tela ***wave*** selecione ***clock*** (com os valores padrão) para o relógio usando o botão direito.

Para o relógio selecionar Clock





Pressionar OK

Tarefa

- **Passo 5**: Após a compilação e verificação de que não há erros de sintaxe **simular** o circuito no **ModelSim** para as 4 operações possíveis, 2A, 3A, A+B, A+2B da seguinte forma:
  - Force, por exemplo, A=3, B=$4_{(10)}$ e Sel=$10_{(2)}$ para visualizar a **soma** $3_{(10)}+4_{(10)}=7_{(10)}$, mas primeiro zerando o sistema usando **Reset=1** e **Enter=0** e faça *run*. Deixe de forçar o Reset (faça **Reset=0** e **Enter=1)** e faça *run* novamente, daí a saída deve mudar de $0_{(10)}$ para $7_{(10)}$.

  - Faça a simulação para as seguintes combinações de entrada:
    - A = 3 e B = 4 para as 4 operações possíveis (2A, 3A, A+B e A+2B).
    - A = 2 e B = 7 para as 4 operações possíveis (2A, 3A, A+B e A+2B).

  - **Passo 6:** Visualize o diagrama de blocos gerado **Tools-> Netlist Viewers-> RTL Viewer.**

Introdução
Tarefa

Emulação na placa DE2

Emulação na placa DE2

Passo 7: Fazer a emulação na placa DE2 do **Aritmetico_reg**. Os passos para a emulação são:

7.1: A *entity* deve ser (**ou usar Mapper no emulador online**):

OBS: Modificar VHDL para mostrar operandos A e B em HEX3 e HEX2, respectivamente.

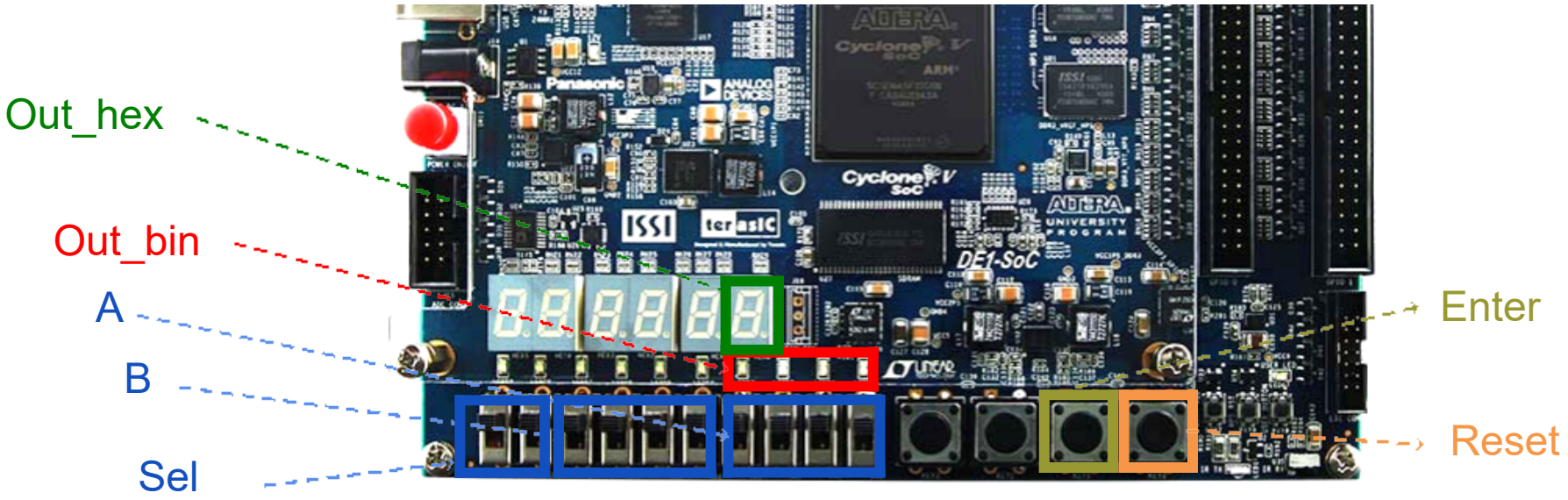
```
entity  Aritmetico_reg is
port (
CLK_500Hz:          in    std_logic;                        -- CLK
KEY:                in    std_logic_vector(1 downto 0);     -- enter, reset
SW:                 in    std_logic_vector(9 downto 0);     -- C, A, B
LEDR:               out   std_logic_vector(3 downto 0);
HEX0, HEX2, HEX3:   out   std_logic_vector(6 downto 0)
      );
end  Aritmetico_reg;
```

Não se esqueçam de trocar!

```
architecture circuito of  Aritmetico_reg is
...
end circuito;
```

Tarefa

- **Passo 7.2:** Os push buttons (KEY), **quando pressionados**, fornecem **'0'.**
  Dica: Modificar VHDL para aceitar Enable em nível lógico '0'



Emulação na placa DE2

Passo 7: Fazer a emulação na placa DE2 do circuito **aritmético_reg** usando o **emulador on-line** .

7.3 (Emulador on-line): Com as devidas adaptações feitas nos arquivos abra o emulador https://fpgaemu.sites.ufsc.br/login e faça ***UPLOAD*** dos arquivos VHDL que deseja utilizar na emulação; compile.

7.4: Verifique o funcionamento do circuito usando A = 2 e B = 7 para as 4 operações possíveis (2A, 3A, A+B e A+2B), mostre também o funcionamento do reset.